HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ASSIGNMENT 06

MÔN HỌC: PHÁT TRIỂN HỆ THỐNG THÔNG MINH

Lớp D23CTPM01

Giảng viên: Trần Đình Quế

Họ và Tên	**Dương Trọng Thắng**
Mã sinh viên	**B23DCCN752**

Hà Nội – 2026

Mục lục

Thông tin dự án **1**

1 Cơ Sở Lý Thuyết Về RNN **2**
1.1 Chuỗi thời gian và cửa sổ trượt 2
1.2 Vanilla RNN và hidden state 2
1.3 Hàm mất mát và BPTT 2
1.4 Vanishing và exploding gradient 3
1.5 LSTM và GRU 3

2 Thiết Kế Thực Nghiệm **4**
2.1 Dataset và nhiệm vụ 4
2.2 Split theo thời gian và chống data leakage 4
2.3 Kiến trúc và cấu hình huấn luyện 4
2.4 Metric đánh giá 5

3 Thực Nghiệm Trên Amazon Stock Price **6**
3.1 Khảo sát và chất lượng dữ liệu 6
3.2 Tạo sequence và huấn luyện 7
3.3 Kết quả benchmark 7

4 Thực Nghiệm Trên Avocado Price **9**
4.1 Đặc điểm panel data 9
4.2 Tạo sequence theo nhóm và huấn luyện 10
4.3 Kết quả benchmark 10

5 Triển Khai Mô Hình Trên Localhost **12**
5.1 Kiến trúc deploy 12
5.2 Đầu vào và API 12
5.3 Giao diện Amazon 13
5.4 Giao diện Avocado 14
5.5 Cách chạy 14

6 Tổng Hợp Và Kết Luận **15**
6.1 Tổng hợp kết quả 15
6.2 Kiểm toán khả năng tái lập 15
6.3 Giới hạn và hướng mở rộng 15
6.4 Kết luận 16

Danh mục hình ảnh

1.1 Minh họa sự suy giảm của tích đạo hàm tanh theo số bước BPTT 3

3.1 EDA tổng quan lịch sử giá, phân phối Close, biến động ngày và volume Amazon 6
3.2 Rolling statistics, correlation OHLCV và lợi suất năm của Amazon 7
3.3 Learning curves, actual–predicted và residual trên test Amazon 8

4.1 Phân phối giá, xu hướng tuần, volume và so sánh region của Avocado 9
4.2 Correlation, quan hệ giá–volume, seasonality và biến thiên theo nhóm 10
4.3 Learning curves, dự báo TotalUS–conventional và residual Avocado 11

5.1 Giao diện localhost Amazon sau khi dự báo bằng PyTorch 13
5.2 Giao diện localhost Avocado cho TotalUS–conventional sau khi dự báo bằng PyTorch 14

Danh mục bảng biểu

2.1 Tổng quan hai dataset của Assignment 06 4
2.2 Số mẫu train, validation và test 4
2.3 Cấu hình huấn luyện chung 5

3.1 Kết quả test trên Amazon Stock Price 7

4.1 Kết quả test tổng hợp trên 108 chuỗi Avocado 10

5.1 Đầu vào của hai ứng dụng deploy 12

6.1 Mô hình có RMSE thấp nhất trong từng dataset 15
6.2 Checklist tái lập Assignment 06 15

Thông tin dự án

GitHub project: `https://github.com/duongthang2710/PTHTTM`

Assignment 06 được triển khai trên hai bài toán dự báo giá độc lập. Mã nguồn thực nghiệm nằm tại:

- Amazon Stock Price:
  `Assignment6/Amazon Stock Price/notebook/amazon_rnn_forecasting.ipynb`;
- Avocado Price:
  `Assignment6/Avocado Price/notebook/avocado_rnn_forecasting.ipynb`.

Mỗi notebook thực hiện trọn quy trình từ khảo sát dữ liệu, tạo cửa sổ thời gian, huấn luyện vanilla RNN bằng PyTorch và Keras đến đánh giá trên tập test theo thứ tự thời gian. Các trọng số tốt nhất cùng tham số chuẩn hóa được lưu để hai ứng dụng Flask có thể suy luận trực tiếp trên localhost. Hình minh họa và số liệu trong báo cáo được lấy từ kết quả đã lưu của chính các notebook này.

Chương 1

Cơ Sở Lý Thuyết Về RNN

1.1 Chuỗi thời gian và cửa sổ trượt

Gọi $p_1, p_2, \ldots, p_T$ là các mức giá đã được sắp theo thời gian. Để đưa chuỗi này vào mô hình học có giám sát, tại mỗi vị trí ta dùng L giá liên tiếp làm đầu vào và giá ngay sau cửa sổ làm nhãn:

$$\mathbf{x}^{(i)} = [p_i, p_{i+1}, \ldots, p_{i+L-1}], \qquad y^{(i)} = p_{i+L}. \tag{1.1}$$

Trong phần Amazon, $L = 60$ tương ứng 60 phiên giao dịch; với Avocado, $L = 12$ tương ứng 12 tuần của cùng một cặp region–type. Cách tạo mẫu này cho phép các cửa sổ chồng lấn nhưng vẫn bảo đảm nhãn luôn xuất hiện sau toàn bộ dữ liệu đầu vào.

1.2 Vanilla RNN và hidden state

Vanilla RNN duy trì một trạng thái ẩn để tóm tắt thông tin đã đi qua. Ở bước t, quan sát mới và trạng thái của bước trước được kết hợp theo các phương trình:

$$\mathbf{a}_t = \mathbf{x}_t W_{xh} + \mathbf{h}_{t-1} W_{hh} + \mathbf{b}_h, \tag{1.2}$$

$$\mathbf{h}_t = \tanh(\mathbf{a}_t), \tag{1.3}$$

$$\hat{y} = W_{hy}\mathbf{h}_L + b_y. \tag{1.4}$$

W_{xh} biến đổi quan sát hiện tại, còn W_{hh} quyết định phần thông tin được truyền dọc chuỗi. Các trọng số được dùng lại ở mọi thời điểm nên độ dài lookback không làm tăng số tham số. Hai notebook đều theo cấu hình many-to-one: mỗi cửa sổ chỉ sinh ra một dự báo cho mốc kế tiếp.

Với batch, input PyTorch có dạng (B, L, d), hidden state có dạng (n_{layer}, B, H). Trong thực nghiệm, $d = 1$, $H = 64$, $n_{layer} = 1$ và đầu ra có một phần tử. Keras có 4,289 tham số; PyTorch có 4,353 tham số do `nn.RNN` tách recurrent bias thành hai vector.

1.3 Hàm mất mát và BPTT

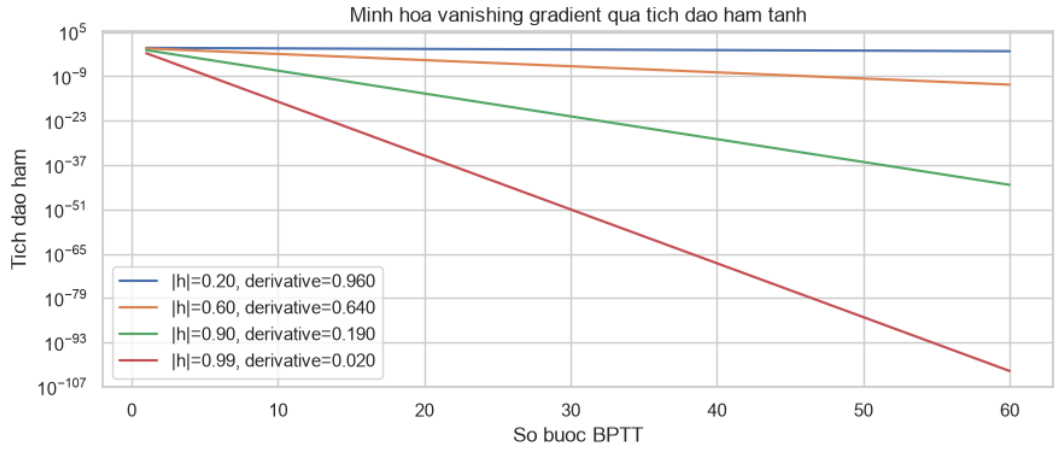
Bài toán hồi quy tối ưu Mean Squared Error:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2. \tag{1.5}$$

Backpropagation Through Time (BPTT) unroll mạng qua L bước, truyền gradient từ output về hidden state cuối rồi tiếp tục về các hidden state trước đó. Adam sử dụng moment bậc một và bậc hai để cập nhật tham số; `ReduceLROnPlateau` giảm learning rate khi validation loss chững.

1.4 Vanishing và exploding gradient

Gradient qua nhiều bước chứa tích các Jacobian $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$. Nếu chuẩn tích nhỏ hơn 1, gradient biến mất; nếu lớn hơn 1, gradient bùng nổ. Đạo hàm $1 - \tanh^2(z)$ rất nhỏ khi tanh bão hòa gần -1 hoặc 1.



Hình 1.1: Minh họa sự suy giảm của tích đạo hàm tanh theo số bước BPTT

Notebook giảm rủi ro bằng chuẩn hóa input, cửa sổ hữu hạn và gradient clipping với chuẩn tối đa 1.0 trong PyTorch. Dropout 0.2 regularize hidden representation; early stopping chọn weights tốt nhất.

1.5 LSTM và GRU

LSTM bổ sung cell state cùng các cổng input, forget và output để kiểm soát thông tin dài hạn. GRU gọn hơn với update gate và reset gate. Hai biến thể thường giữ phụ thuộc xa tốt hơn vanilla RNN, nhưng Assignment 06 dùng vanilla RNN để biểu diễn trực tiếp phương trình hồi quy cơ bản và giữ hai framework tương đương.

Chương 2

Thiết Kế Thực Nghiệm

2.1 Dataset và nhiệm vụ

Bảng 2.1: Tổng quan hai dataset của Assignment 06

Dataset	Số dòng	Khoảng thời gian	Target	Lookback	Tần suất
Amazon Stock Price	6,684	15/05/1997–05/12/2023	Close	60	Phiên giao dịch
Avocado Price	18,249	04/01/2015–25/03/2018	AveragePrice	12	Tuần

Amazon là một chuỗi duy nhất. Avocado là panel data gồm 54 region, hai type và 108 chuỗi độc lập. Mục tiêu của cả hai là dự báo một bước: phiên giao dịch kế tiếp hoặc tuần kế tiếp.

2.2 Split theo thời gian và chống data leakage

Toàn bộ mẫu được sắp theo thời điểm của target rồi chia liên tiếp theo tỷ lệ xấp xỉ 80%–10%–10%. Do không trộn ngẫu nhiên, tập validation và test luôn nằm sau tập train, đúng với tình huống dự báo dữ liệu chưa xuất hiện trong thực tế.

Bảng 2.2: Số mẫu train, validation và test

Dataset	Train	Validation	Test
Amazon	5,299	662	663
Avocado	13,498	1,727	1,728

`StandardScaler` chỉ học tham số trên phần train. Amazon cần một scaler cho chuỗi duy nhất; Avocado dùng scaler riêng cho từng cặp region–type vì mặt bằng giá giữa các thị trường khác nhau. Khi xử lý validation và test, notebook tái sử dụng mean và standard deviation của train thay vì fit lại.

2.3 Kiến trúc và cấu hình huấn luyện

Cả PyTorch và Keras dùng cùng pipeline:

$$\text{Sequence}(L, 1) \rightarrow \text{RNN}(64, \tanh) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(1). \tag{2.1}$$

Bảng 2.3: Cấu hình huấn luyện chung

Thành phần	**Giá trị**
Optimizer / loss	Adam 10^{-3} / MSE
Epoch	Tối đa 250, tối thiểu 30
Batch size	64
Early stopping	Patience 25 theo validation loss
Scheduler	Factor 0.5, patience 5
Regularization	Dropout 0.2, gradient clipping 1.0
Random seed	42
PyTorch device	MPS nếu khả dụng, nếu không dùng CPU

2.4 Metric đánh giá

Các metric được tính sau khi inverse-transform về đơn vị giá gốc:

$$\text{MAE} = \frac{1}{N}\sum_i |y_i - \hat{y}_i|, \tag{2.2}$$

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_i (y_i - \hat{y}_i)^2}, \tag{2.3}$$

$$\text{MAPE} = \frac{100}{N}\sum_i \left|\frac{y_i - \hat{y}_i}{y_i}\right|, \tag{2.4}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{2.5}$$

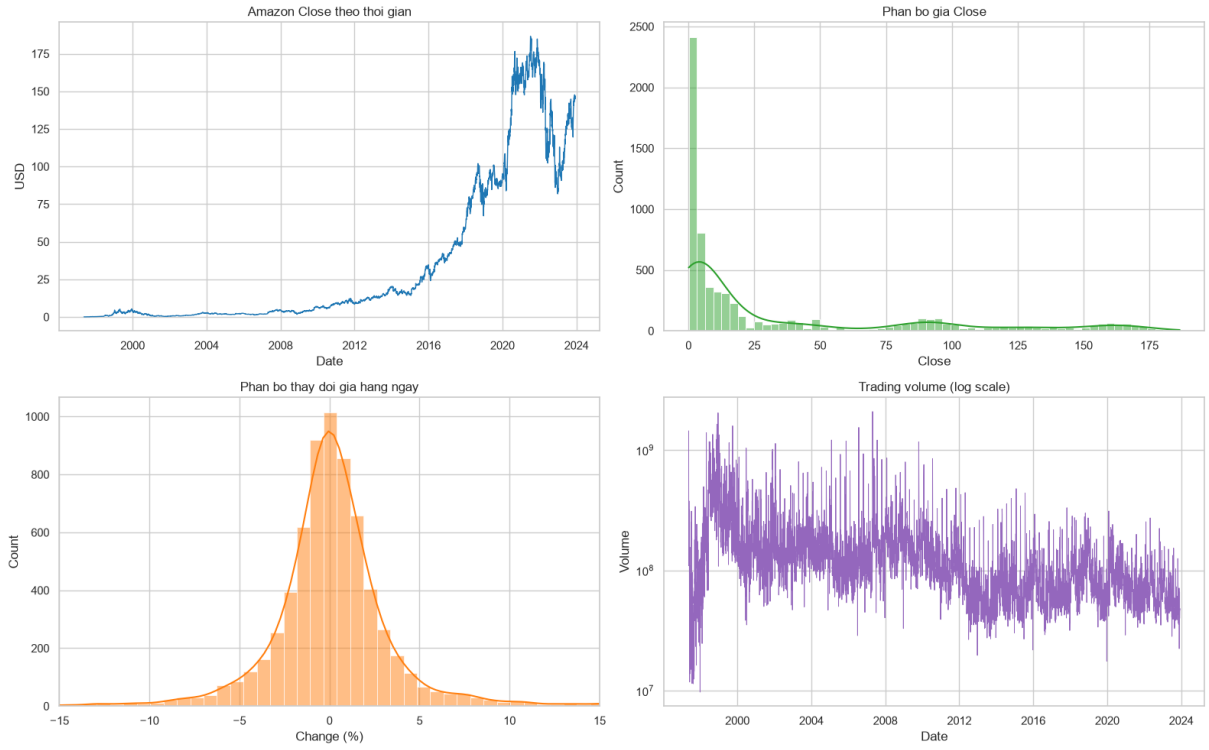
Persistence baseline dùng giá cuối cửa sổ làm giá dự báo và là đối chứng quan trọng cho time series.

Chương 3

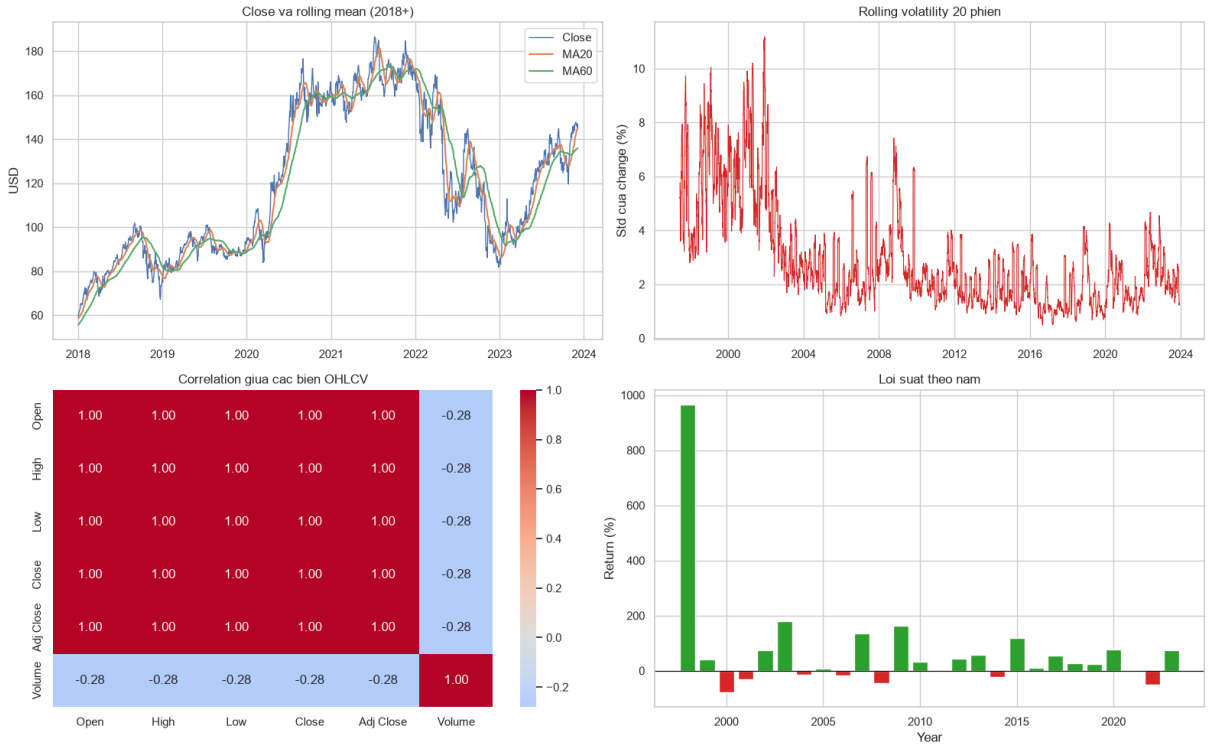
Thực Nghiệm Trên Amazon Stock Price

3.1 Khảo sát và chất lượng dữ liệu

Tệp Amazon gồm Date, Open, High, Low, Close, Adj Close và Volume; bước kiểm tra không phát hiện giá trị khuyết hay bản ghi trùng. Close tăng theo xu hướng dài hạn, còn mức thay đổi theo ngày tập trung quanh 0 và có một số biến động lớn. Dù ngày lịch không cách đều vì cuối tuần và ngày nghỉ, thứ tự dòng vẫn biểu diễn đúng các phiên giao dịch liên tiếp.



Hình 3.1: EDA tổng quan lịch sử giá, phân phối Close, biến động ngày và volume Amazon



Hình 3.2: Rolling statistics, correlation OHLCV và lợi suất năm của Amazon

Các biến OHLC tương quan cao vì cùng phản ánh diễn biến trong một phiên. Close cũng có tự tương quan lớn, song kết quả này chịu ảnh hưởng đáng kể của xu hướng và không đủ để kết luận chuỗi dừng. Đồ thị rolling volatility cho thấy mức rủi ro thay đổi theo từng giai đoạn thị trường.

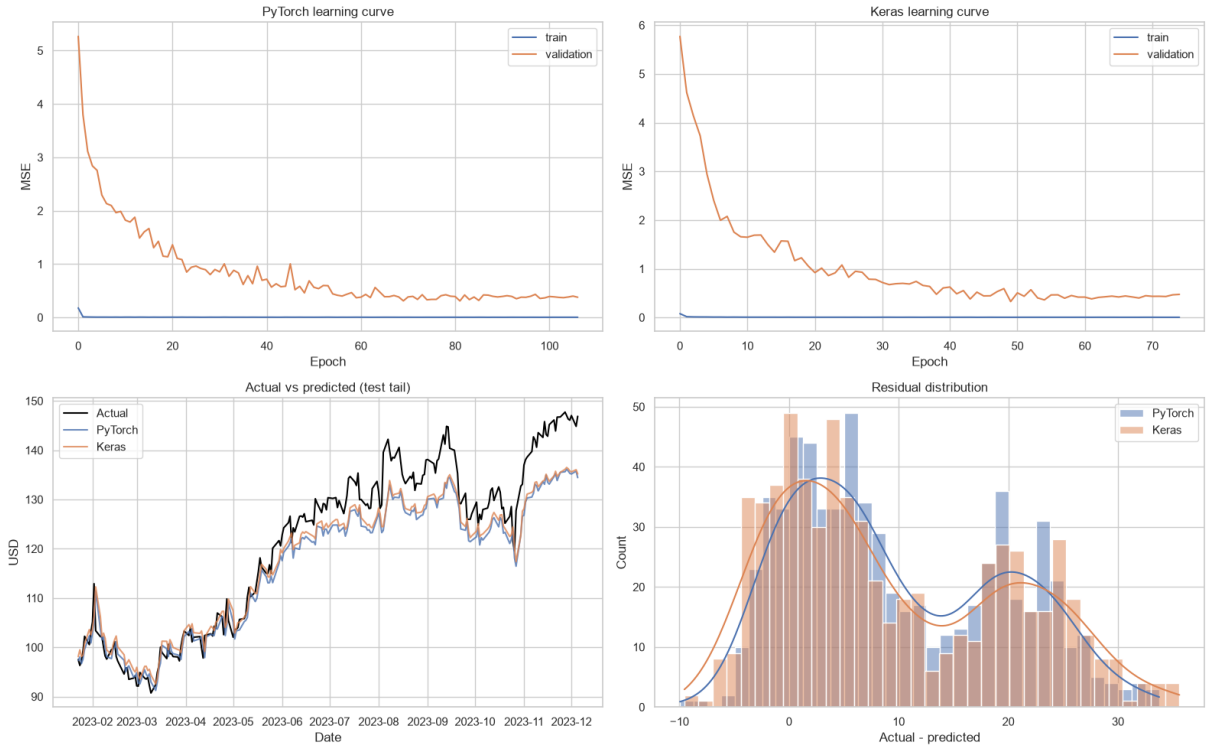
3.2 Tạo sequence và huấn luyện

Sau lookback 60, dataset tạo 6,624 samples. Scaler train-only có mean 11.5805 và scale 17.4447. PyTorch chạy 107 epoch, mất 101.50 giây và lưu best validation loss 0.309872. Keras chạy 75 epoch, mất 18.88 giây. Cả hai best weights được nạp lại độc lập trước khi test.

3.3 Kết quả benchmark

Bảng 3.1: Kết quả test trên Amazon Stock Price

Mô hình	**MAE**	**RMSE**	**MAPE (%)**	R^2	**Train (s)**
Persistence baseline	**2.2720**	**3.1507**	**1.7519**	**0.9874**	0.00
PyTorch RNN	10.4466	13.6738	6.7825	0.7633	101.50
Keras RNN	10.5704	14.1749	6.8377	0.7456	18.88



Hình 3.3: Learning curves, actual–predicted và residual trên test Amazon

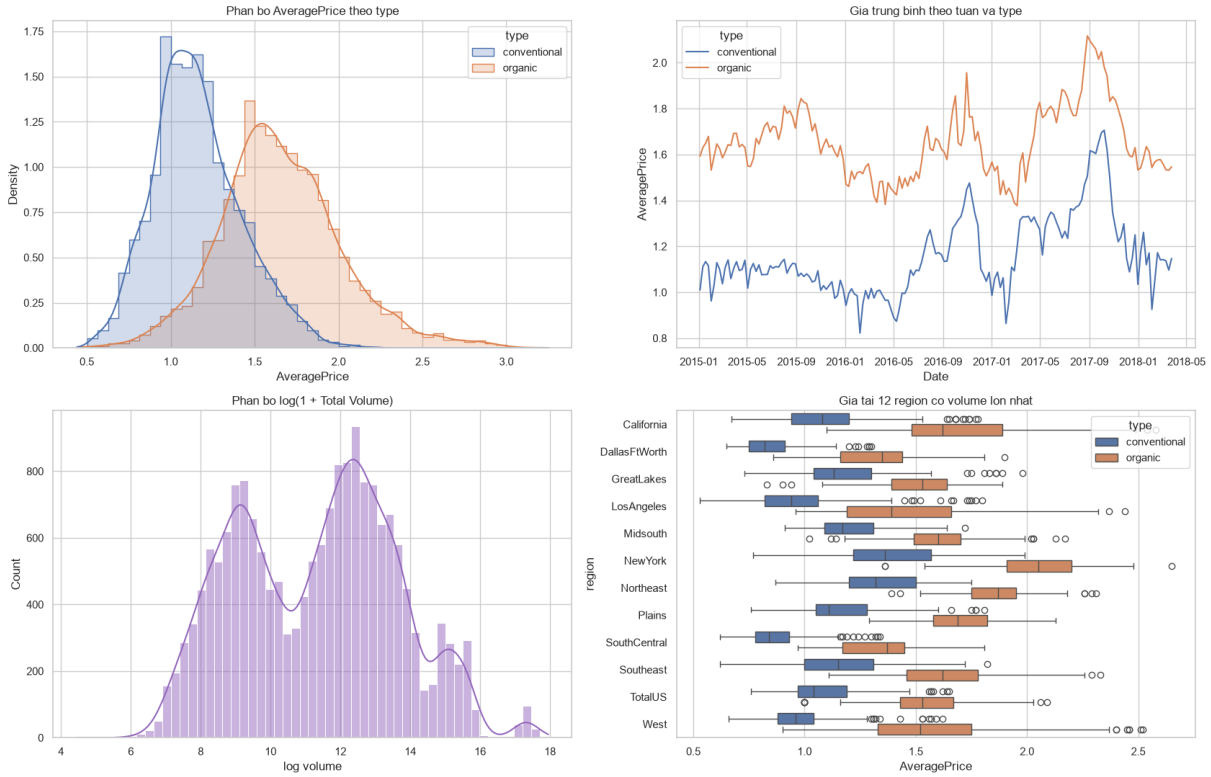
Trên Amazon, quy tắc persistence đạt sai số thấp hơn cả hai RNN. Nguyên nhân hợp lý là mô hình đang học trực tiếp mức giá có xu hướng mạnh, trong khi phần test nằm ở vùng giá cao hơn phần lớn dữ liệu train. RNN dùng tanh vì vậy dễ kéo dự báo về miền đã quan sát; baseline lại khai thác trực tiếp tính liên tục giữa hai phiên liền nhau. Có thể cải thiện thí nghiệm bằng return/difference, rolling normalization hoặc cơ chế walk-forward retraining.

Chương 4

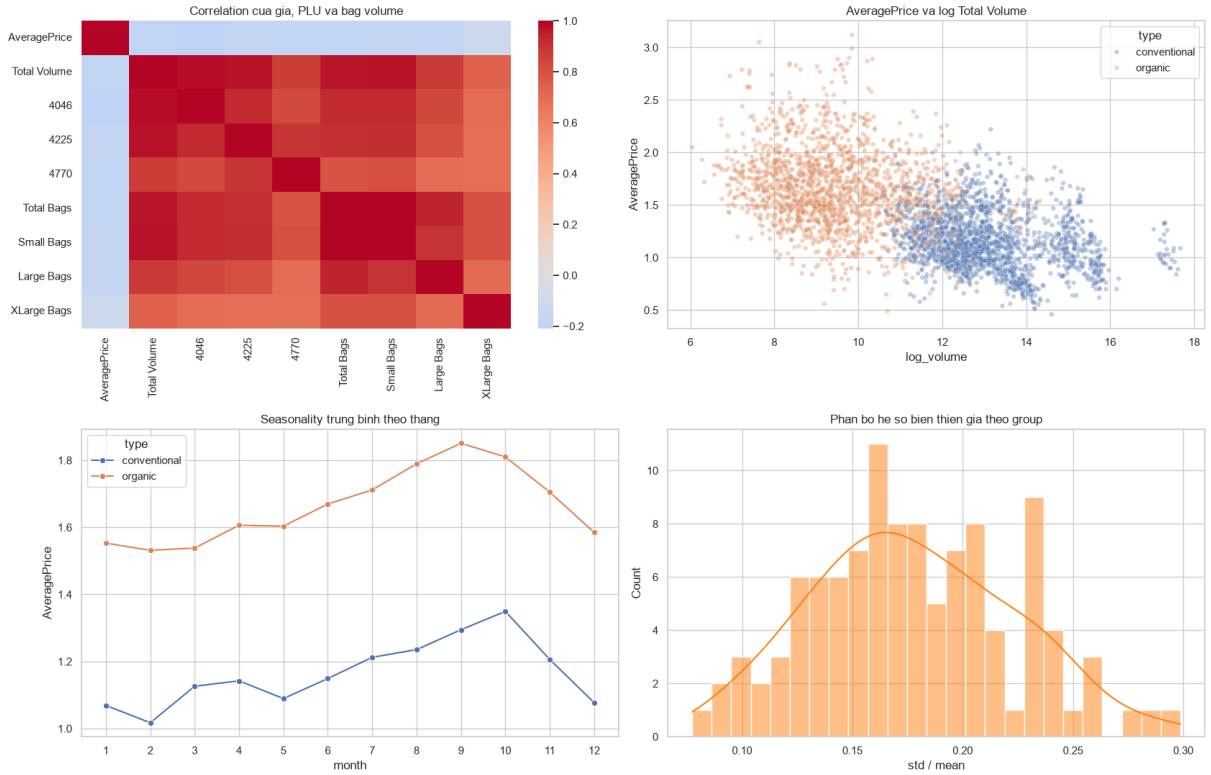
Thực Nghiệm Trên Avocado Price

4.1 Đặc điểm panel data

Bộ dữ liệu Avocado có 18,249 dòng, trải trên 169 mốc tuần, 54 region và hai loại conventional/organic. Tổ hợp (Date, region, type) không trùng lặp; 107 chuỗi đủ 169 quan sát, chuỗi còn lại có 166. Nhóm organic thường có AveragePrice cao hơn conventional, còn volume phân bố lệch phải và chênh lệch đáng kể giữa vùng.



Hình 4.1: Phân phối giá, xu hướng tuần, volume và so sánh region của Avocado



Hình 4.2: Correlation, quan hệ giá–volume, seasonality và biến thiên theo nhóm

Seasonality cho thấy mức giá thay đổi theo tháng và type. Correlation giữa các cột volume/PLU/bag phản ánh cấu trúc tổng hợp của dataset; scatter giá–log volume chỉ mang ý nghĩa mô tả, không chứng minh quan hệ nhân quả.

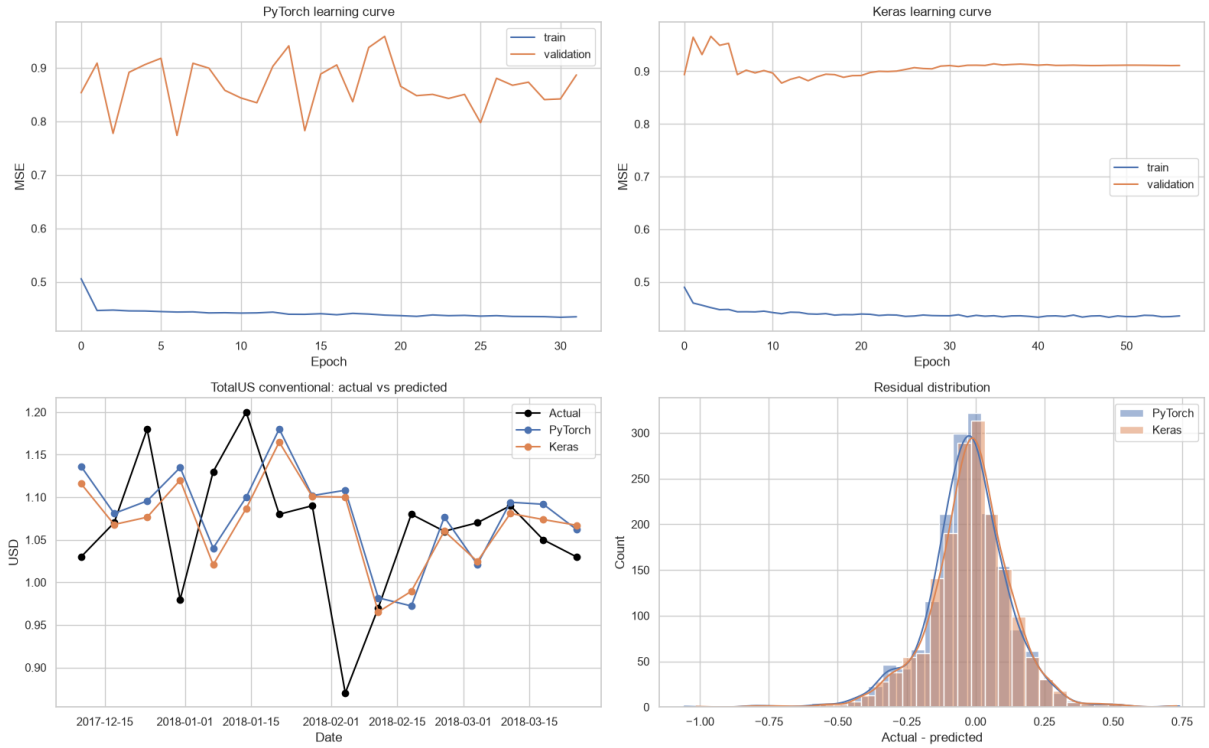
4.2 Tạo sequence theo nhóm và huấn luyện

Notebook tạo cửa sổ bên trong từng chuỗi region–type trước khi gộp mẫu, nhờ đó không có input nào nối giá của hai thị trường khác nhau. Một RNN chung được huấn luyện trên dữ liệu của cả 108 nhóm. Bản PyTorch dừng sau 32 epoch, mất 22.72 giây với validation loss tốt nhất 0.773740; bản Keras mất 10.88 giây và đạt validation loss tốt nhất 0.877579.

4.3 Kết quả benchmark

Bảng 4.1: Kết quả test tổng hợp trên 108 chuỗi Avocado

Mô hình	**MAE**	**RMSE**	**MAPE (%)**	R^2	**Train (s)**
Keras RNN	**0.1099**	**0.1534**	**8.7926**	**0.7490**	10.88
PyTorch RNN	0.1122	0.1562	9.0260	0.7398	22.72
Persistence baseline	0.1199	0.1742	9.5478	0.6762	0.00



Hình 4.3: Learning curves, dự báo TotalUS–conventional và residual Avocado

Ở Avocado, cả hai RNN đều cải thiện bốn metric so với persistence. Keras dẫn đầu với RMSE 0.1534 và $R^2 = 0.7490$, nhưng khoảng cách với PyTorch không lớn. Sai khác còn lại có thể xuất phát từ phép khởi tạo, cách xử lý recurrent bias và quỹ đạo tối ưu riêng của từng framework dù cấu hình tổng thể tương đương.

Chương 5

Triển Khai Mô Hình Trên Localhost

5.1 Kiến trúc deploy

Mỗi bài toán có một `deploy.py` độc lập, bao gồm Flask app, giao diện HTML/CSS/JavaScript, định nghĩa lại kiến trúc RNN và phần nạp trọng số. Một request dự báo đi qua chuỗi xử lý:

$$\text{Web input} \rightarrow \text{Validate} \rightarrow \text{Scale} \rightarrow \text{RNN inference} \rightarrow \text{Inverse scale} \rightarrow \text{JSON/UI result.} \tag{5.1}$$

Mô hình được cache sau lần nạp đầu tiên để các request tiếp theo không phải đọc lại weights. PyTorch chọn MPS khi thiết bị hỗ trợ và chuyển về CPU trong trường hợp còn lại; Keras dùng thiết bị do TensorFlow phát hiện. Cả hai server mặc định chỉ bind `127.0.0.1` nên chỉ phục vụ trên máy đang chạy.

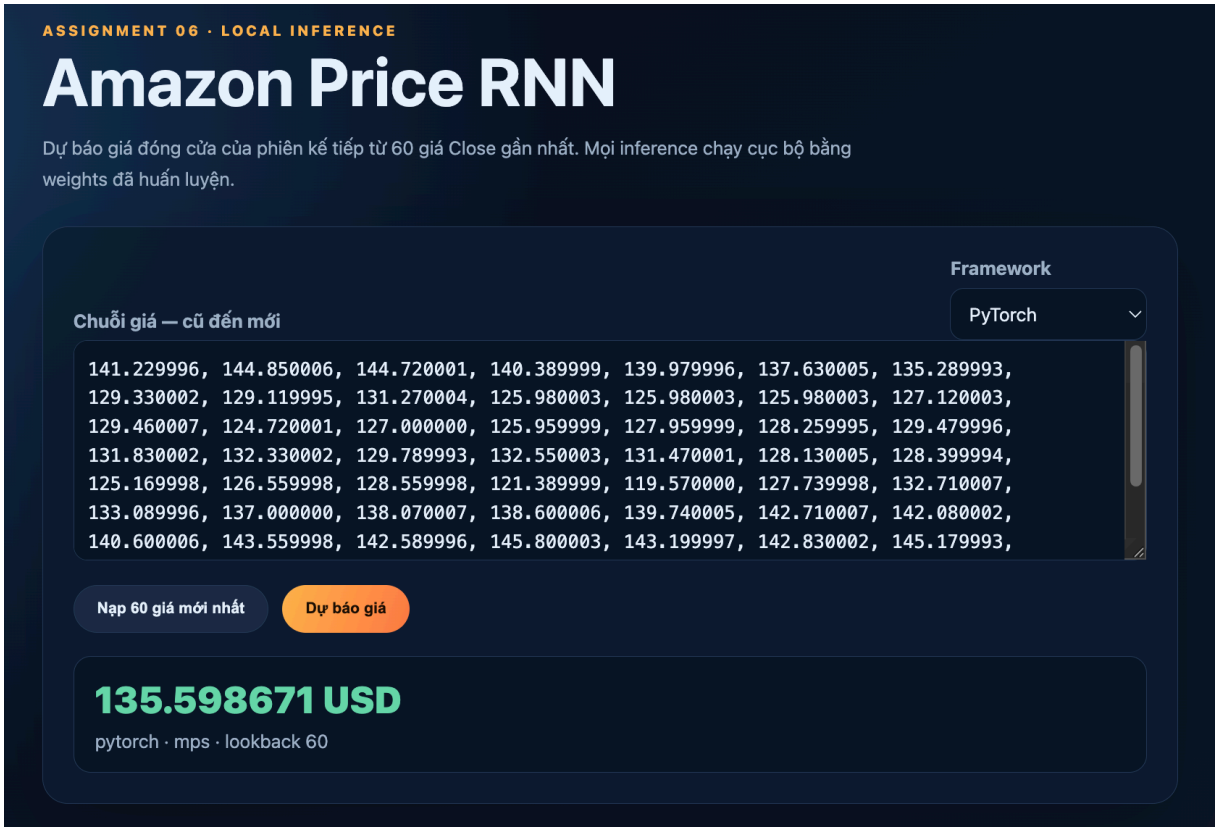
5.2 Đầu vào và API

Bảng 5.1: Đầu vào của hai ứng dụng deploy

Ứng dụng	Framework	Chuỗi giá	Thông tin thêm	Đầu ra
Amazon	pytorch / keras	60 giá Close, cũ → mới	Không	Close phiên kế tiếp
Avocado	pytorch / keras	12 AveragePrice, cũ → mới	region, type	AveragePrice tuần kế tiếp

Amazon cung cấp `POST /api/predict`. Avocado có thêm `GET /api/latest` để đọc 12 tuần mới nhất của nhóm được chọn. Backend từ chối sai lookback, giá âm, số không hữu hạn, framework hoặc region–type không hợp lệ.

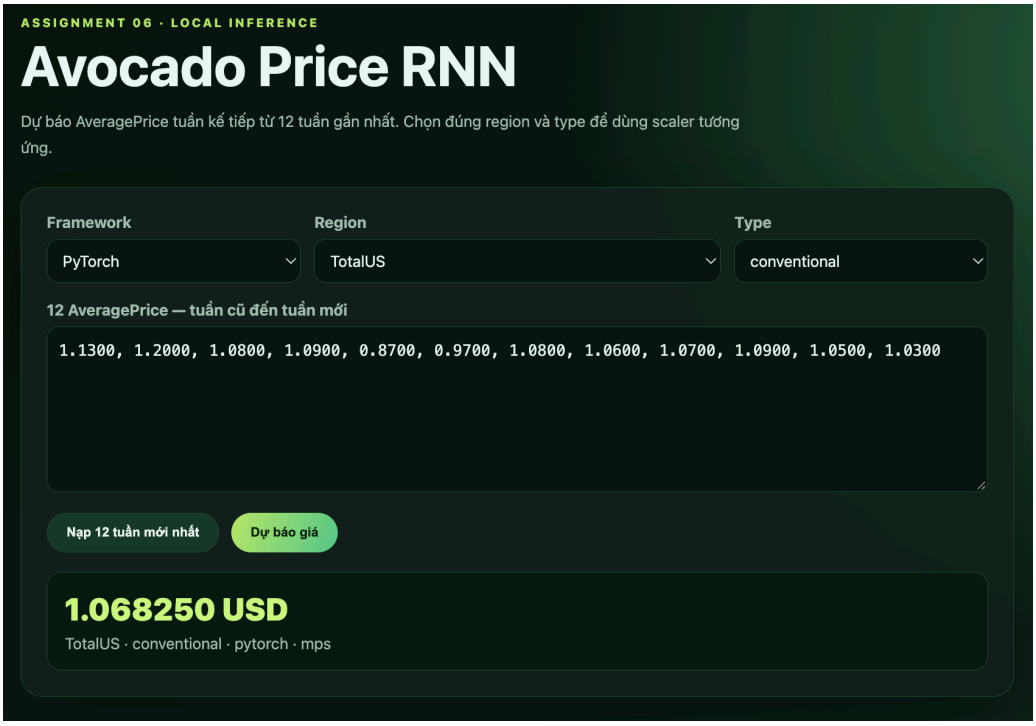
5.3 Giao diện Amazon



Hình 5.1: Giao diện localhost Amazon sau khi dự báo bằng PyTorch

Giao diện tự nạp 60 giá Close cuối từ `AMZN.csv`, cho phép đổi framework và hiển thị giá dự báo cùng device inference. Cổng mặc định là 8001.

5.4 Giao diện Avocado



Hình 5.2: Giao diện localhost Avocado cho TotalUS–conventional sau khi dự báo bằng PyTorch

Giao diện cho phép chọn 54 region, hai type và framework. Khi region/type thay đổi, 12 tuần gần nhất được nạp lại theo nhóm. Cổng mặc định là 8002.

5.5 Cách chạy

Từ thư mục gốc project, kích hoạt môi trường và chạy hai terminal:

```
conda activate ISD
    python "Assignment6/Amazon Stock Price/deploy.py" --web
    python "Assignment6/Avocado Price/deploy.py" --web
```

Đoạn mã 5.1: Khởi động hai web deploy trên localhost

Sau đó mở một trong hai địa chỉ sau:

- Amazon: `http://127.0.0.1:8001`;
- Avocado: `http://127.0.0.1:8002`.

Dừng server bằng `Ctrl+C`. CLI vẫn được giữ để tích hợp tự động hoặc kiểm thử không giao diện.

Chương 6

Tổng Hợp Và Kết Luận

6.1 Tổng hợp kết quả

Bảng 6.1: Mô hình có RMSE thấp nhất trong từng dataset

Dataset	Mô hình	MAE	RMSE	MAPE (%)	R^2
Amazon	Persistence baseline	2.2720	3.1507	1.7519	0.9874
Avocado	Keras RNN	0.1099	0.1534	8.7926	0.7490

Hai dataset cho hai kết luận khác nhau. Với Amazon, persistence là đối chứng bắt buộc vì một mô hình phức tạp chưa chắc vượt quy tắc giữ nguyên giá gần nhất. Với Avocado, RNN huấn luyện chung sau khi chuẩn hóa riêng từng nhóm đã học được biến động theo tuần và tạo ra cải thiện thực sự so với baseline.

6.2 Kiểm toán khả năng tái lập

Bảng 6.2: Checklist tái lập Assignment 06

Hạng mục	Cách thực hiện	Ý nghĩa
Random seed	Seed 42 cho NumPy, Python, PyTorch, Keras	Giảm dao động khởi tạo
Split	Theo target time 80/10/10, không shuffle giữa tập	Không nhìn tương lai
Scaling	Fit chỉ trên train; Avocado theo nhóm	Chống data leakage
Thiết bị	MPS ưu tiên, CPU fallback	Tận dụng Apple Silicon
Checkpoint	Lưu best validation loss	Deploy weights tốt nhất
Reload test	Dự đoán sau reload khớp notebook	Xác nhận artifact dùng được
Baseline	Persistence trên cùng test set	Đối chứng thực tế
Deploy test	Flask page/API và lỗi input	Kiểm chứng inference end-to-end

6.3 Giới hạn và hướng mở rộng

- Amazon dùng mức giá chưa dừng và scaler cố định từ train; nên thử return, difference, walk-forward validation và rolling normalization.

- Vanilla RNN có hạn chế về phụ thuộc dài; có thể benchmark thêm LSTM, GRU và attention.
- Hai mô hình chỉ dùng lịch sử giá; có thể bổ sung volume, OHLCV, calendar features và biến ngoại sinh.
- Cần chạy nhiều seed và báo cáo mean $\pm$ standard deviation để đánh giá độ ổn định.
- Flask development server phù hợp localhost; triển khai nhiều người dùng cần WSGI server, authentication, logging và giới hạn request.

6.4 Kết luận

Assignment 06 đã hoàn thiện quy trình dự báo một bước bằng vanilla RNN cho một chuỗi đơn và dữ liệu panel. Hai phiên bản PyTorch/Keras được đánh giá bằng split thời gian, lưu checkpoint tốt nhất và tái sử dụng trong ứng dụng web localhost. Kết quả Avocado cho thấy RNN có thể vượt persistence khi dữ liệu được tách và scale đúng theo nhóm; trường hợp Amazon lại chỉ ra rằng cách biểu diễn chuỗi và baseline có ảnh hưởng lớn hơn độ phức tạp của mô hình. Notebook, weights, metadata, chế độ CLI và Flask app hiện liên kết thành một quy trình nhất quán từ phân tích dữ liệu đến suy luận.